SENAI — Avaliação Formativa
Desenvolvimento de Aplicação Mobile com Flutter

Mapa de Locais

Documentação Técnica

Aplicação de localização em Flutter com Firebase Cloud Firestore:
funcionalidades, conexões e funcionamento interno

Aluno: Gabriel Xavier
Repositório: github.com/GabrielXavier125/Mobile
Projeto Firebase: mapa-locais-gabriel

Outubro de 2026 — versão 1.0

Sumário

1. Introdução 4
   1.1 Problemática 4
   1.2 Objetivo 4
   1.3 Escopo 4
   1.4 Sobre este documento 4
2. Visão geral da solução 5
   2.1 Tecnologias e versões 5
   2.2 Projeto Firebase 6
3. Funcionalidades 7
   3.1 Tela 1 — Mapa principal 7
   3.2 Tela 2 — Resultados 8
   3.3 Tela 3 — Detalhes do local 9
   3.4 Cadastro e edição de locais 10
   3.5 Exclusão com confirmação 11
   3.6 Favoritos e Perfil 11
   3.7 Fluxo de navegação 12
   3.8 Mensagens e tratamento de erros 12
   3.9 Requisitos funcionais 12
4. Conexões 14
   4.1 Conexão com o Firebase 14
   4.2 Leitura dos dados em tempo real 15
   4.3 Gravação dos dados (CRUD) 15
   4.4 Segurança: regras do Firestore 16
   4.5 Mapa: OpenStreetMap 16
   4.6 Integrações com o Android 17
   4.7 Ambiente de desenvolvimento 18
5. Arquitetura e organização do código 19
   5.1 Camadas e responsabilidades 19
   5.2 Modelo de dados — coleção locais 19
   5.3 Pesquisa e filtros 20
6. Testes 21
   6.1 Estratégia 21
   6.2 Testes obrigatórios 21
   6.3 Evidência no Firestore 22
7. Instalação e execução 23
   7.1 Pré-requisitos 23

7.2 Executar o app 23
7.3 Comandos úteis 23
8. Requisitos não funcionais, limitações e conclusão 24
8.1 Requisitos não funcionais atendidos 24
8.2 Limitações e melhorias futuras 24
8.3 Conclusão 24
Referências 25

1. Introdução

1.1 Problemática

Uma empresa deseja disponibilizar um aplicativo que permita aos usuários localizar serviços e estabelecimentos em diferentes cidades. O usuário precisa selecionar a cidade, pesquisar o que procura, filtrar os resultados, abrir os detalhes de um estabelecimento e marcar locais como favoritos. Além disso, novos locais precisam ser cadastrados e os registros existentes precisam poder ser alterados ou excluídos — portanto os dados não podem ficar fixos no código: eles precisam estar em um banco de dados em nuvem.

1.2 Objetivo

Desenvolver o aplicativo **Mapa de Locais**, um guia de estabelecimentos (academias, restaurantes, hospitais, farmácias, mercados e cafés) com foco em **Limeira (SP)** — a cidade da turma — e também **São Paulo, Campinas e Jundiaí**, exigidas pelo enunciado, com mapa real, pesquisa, filtros, tela de detalhes e favoritos. Todos os dados ficam na coleção `locais` do **Firebase Cloud Firestore**, com operações completas de cadastro, leitura, edição e exclusão (CRUD).

1.3 Escopo

- **Três telas principais**, seguindo o wireframe da avaliação: Mapa, Resultados e Detalhes.
- Cadastro e edição em **Bottom Sheet** e exclusão com **diálogo de confirmação**, sem criar uma quarta tela.
- Banco de dados **Cloud Firestore** configurado com **FlutterFire CLI** e protegido por **regras de segurança**.
- Plataforma alvo: **Android** (testado no emulador Pixel 8, Android 16).
- Testes automatizados de unidade, de widget e de integração, além dos testes obrigatórios T01 a T11.

1.4 Sobre este documento

Este documento descreve o aplicativo do ponto de vista técnico: **o que ele faz** (seção 3), **como se conecta** ao Firebase, ao mapa e ao Android (seção 4), **como o código está organizado** (seção 5), **como foi testado** (seção 6) e **como executá-lo** (seção 7). O código-fonte completo está no repositório indicado na capa, na pasta `Avaliação Formativa/mapa_locais`.

2. Visão geral da solução

O aplicativo é escrito em **Flutter/Dart** e roda no Android. Ele se comunica com três serviços externos: o **Cloud Firestore** (banco de dados), o **OpenStreetMap** (imagens do mapa) e os **aplicativos do próprio Android** (discador e Google Maps). A figura abaixo resume as camadas internas do app e essas conexões.

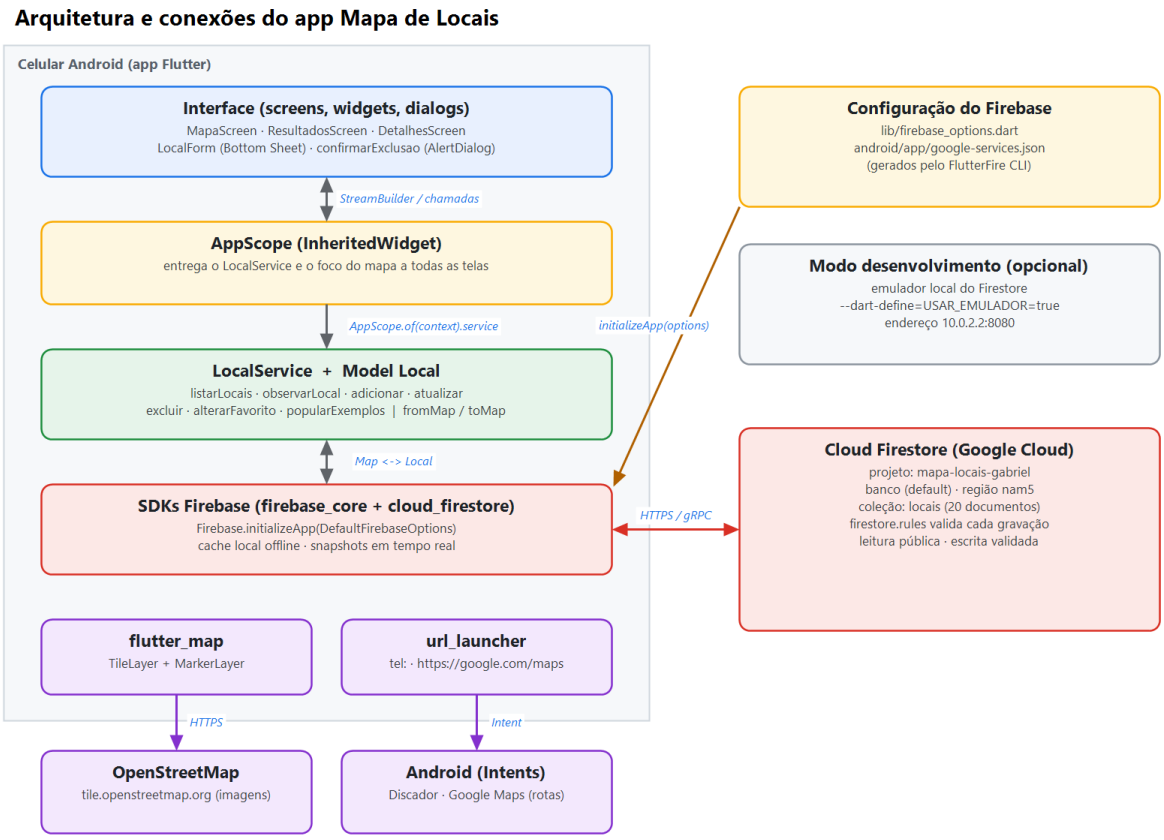


Figura 1 — Arquitetura do aplicativo e suas conexões externas

2.1 Tecnologias e versões

Tecnologia	Versão	Uso no projeto
Flutter / Dart	3.47.6 / 3.13.5	Interface e lógica do aplicativo
firebase_core	4.15.0	Inicialização e conexão com o projeto Firebase
cloud_firestore	6.10.0	Leitura e gravação na coleção `locais`
Firebase CLI / FlutterFire CLI	15.33.0 / 1.4.1	Criação do projeto, regras e geração do `firebase_options.dart`
flutter_map + latlong2	8.3.2 + 0.10.1	Mapa real com marcadores (OpenStreetMap)
url_launcher	6.3.3	Ações "Ligar" e "Como chegar"
fake_cloud_firestore	4.3.0	Firestore em memória para os testes
integration_test	SDK do Flutter	Teste de ponta a ponta no emulador

Tecnologia	Versão	Uso no projeto
Android Gradle Plugin / Gradle	9.1.0 / 9.3.1	Build do aplicativo Android
Android	API 36 (Android 16)	Emulador Pixel 8 usado nos testes

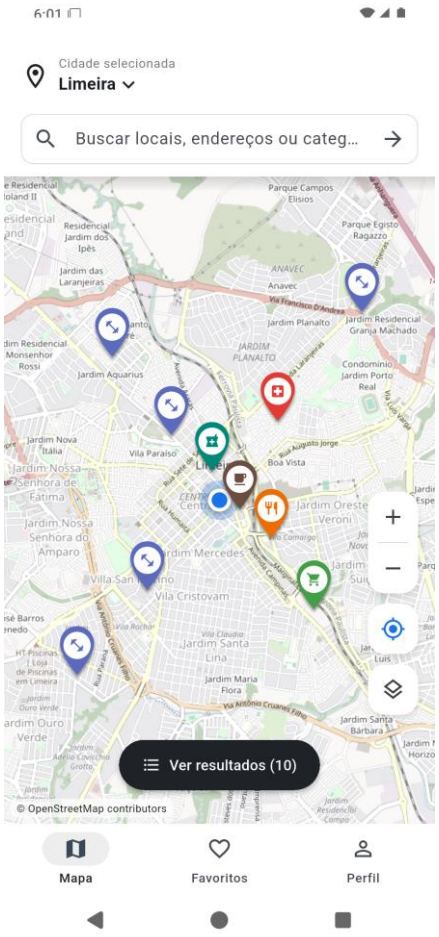
2.2 Projeto Firebase

Item	Valor
Projeto	`mapa-locais-gabriel` (nome de exibição: Mapa Locais Flutter)
Plano	Spark (gratuito)
Banco de dados	Cloud Firestore, modo nativo, banco `(default)`, região `nam5`
Coleção principal	`locais` — 20 documentos de exemplo: 10 em Limeira, 4 em Campinas, 3 em Jundiaí e 3 em São Paulo. Os locais são fictícios (nomes, telefones e avaliações inventados), posicionados em ruas reais.
Aplicativo registrado	Android, pacote `com.exemplo.mapa_locais`
Regras de segurança	Arquivo `firestore.rules`, publicado com `firebase deploy`

3. Funcionalidades

O app possui três telas principais, uma barra de navegação inferior (Mapa, Favoritos e Perfil) e janelas sobrepostas para cadastro, edição, exclusão e escolha de cidade. Todas as informações exibidas vêm do Cloud Firestore em tempo real.

3.1 Tela 1 — Mapa principal

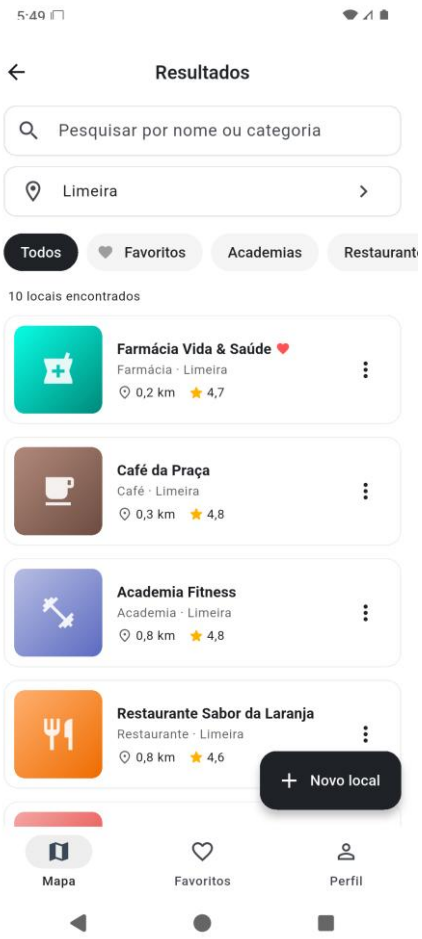


Tela 1 — Mapa principal

Ponto de entrada do aplicativo (`screens/mapa_screen.dart`).

- **Cidade selecionada:** o app abre em **Limeira**. Toque em "Limeira ▾" para escolher entre Limeira, São Paulo, Campinas e Jundiaí; o mapa se desloca para a cidade escolhida.
- **Pesquisa:** o campo "Buscar locais, endereços ou categorias…" abre a Tela 2 já filtrada pelo texto digitado e pela cidade.
- **Mapa real:** imagens do OpenStreetMap no estilo **Suave** (cores leves), com as opções **Colorido** e **Escuro** (modo noturno). O ponto azul pulsante representa a posição do usuário (simulada no centro da cidade).
- **Marcadores:** um pino na cor da categoria para cada local, com o ícone num círculo branco. Ao tocar, o pino cresce, mostra um balão com o nome e um cartão com avaliação, distância e "Ver detalhes".
- **Botões laterais:** zoom (+/−), "Minha localização" (recentraliza o mapa) e "Camadas" (abre o seletor de estilos).
- **Ver resultados (N):** abre a Tela 2 com os locais da cidade.

3.2 Tela 2 — Resultados



Tela 2 — Resultados

Lista os locais lidos do Firestore (`screens/resultados_screen.dart`).

- **Pesquisa** por nome ou categoria, sem diferenciar maiúsculas e acentos ("farmacia" encontra "Farmácia").
- **Filtro de cidade** (inclui "Todas as cidades") e **chips de categoria**: Todos, Favoritos, Academias, Restaurantes, Hospitais, Farmácias, Mercados e Cafés.
- Cada item mostra **imagem, nome, categoria, cidade, distância e avaliação**, ordenados do mais próximo para o mais distante.
- **Tocar no item** abre a Tela 3. O menu ⋮ oferece **Editar** e **Excluir**.
- O botão **Novo local** abre o formulário de cadastro.
- Mostra estados de **carregando**, **nenhum resultado**, **banco vazio** (com botão "Cadastrar locais de exemplo") e **erro de conexão** (com "Tentar novamente").

3.3 Tela 3 — Detalhes do local

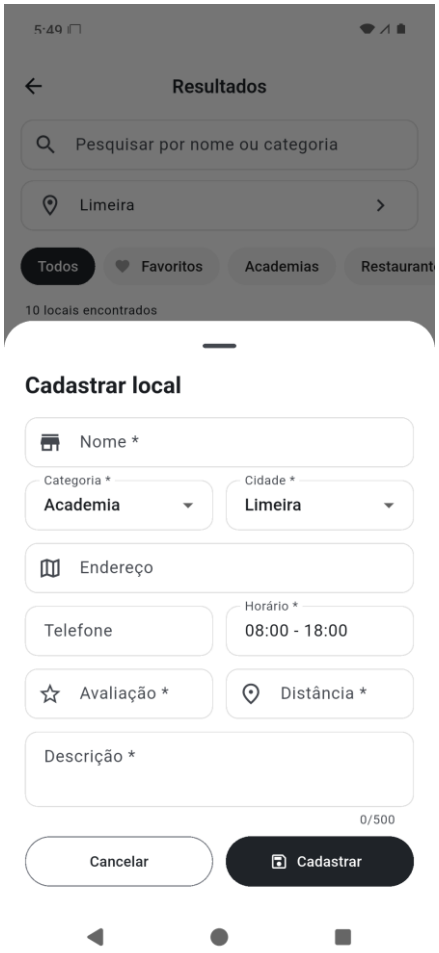


Tela 3 — Detalhes

Recebe o local selecionado e acompanha o documento no Firestore em tempo real (`screens/detalhes_screen.dart`).

- **Galeria** com 5 imagens ilustrativas da categoria (indicador 1/5 e pontos).
- **Nome, categoria, avaliação** (estrelas e nota), **distância** e **horário**, com o selo **Aberto/Fechado** calculado a partir da hora atual.
- **Ações rápidas: Ligar** abre o discador com o telefone do local; **Como chegar** abre a rota no Google Maps; **Agendar** escolhe data e hora da visita.
- **Sobre:** descrição, endereço, cidade, telefone e horário de funcionamento.
- **Coração (favorito):** grava o campo `favorito` no Firestore. **Compartilhar** copia as informações do local.
- **Ver no mapa:** volta à Tela 1 centralizando o local, com o rótulo e o cartão de prévia.

3.4 Cadastro e edição de locais



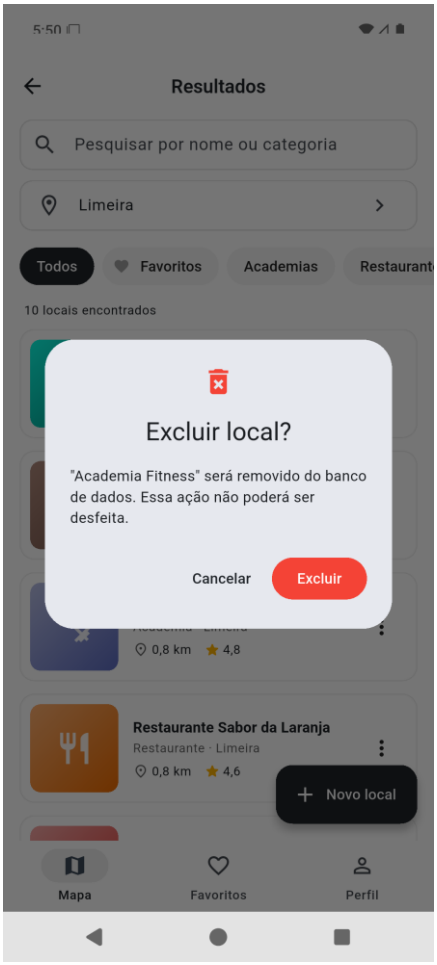
Formulário em Bottom Sheet

O mesmo formulário (`widgets/local_form.dart`) serve para cadastrar (vazio) e editar (preenchido com os dados atuais). Ele abre em um **Bottom Sheet** e valida os campos antes de gravar:

- **Nome:** obrigatório, até 80 caracteres.
- **Categoria e cidade:** escolhidas em listas, o que evita valores inválidos.
- **Horário:** obrigatório, no formato `HH:MM - HH:MM` ou "24 horas".
- **Avaliação:** número de 0 a 5. **Distância:** número de 0 a 500 km. Os dois aceitam vírgula ou ponto.
- **Descrição:** obrigatória, até 500 caracteres. **Endereço e telefone:** opcionais.

No cadastro, a latitude e a longitude do local são geradas perto do centro da cidade escolhida para posicionar o marcador no mapa. Na edição, a posição é mantida, exceto quando a cidade muda.

3.5 Exclusão com confirmação



Confirmação de exclusão

A exclusão **nunca acontece direto ao tocar no botão**. O app abre um `AlertDialog` (`dialogs/confirm_delete.dart`) com o nome do local e o aviso de que a ação não pode ser desfeita.

- **Cancelar:** fecha o diálogo sem alterar nada.
- **Excluir:** remove o documento do Firestore e mostra uma mensagem verde de confirmação. Como a lista usa leitura em tempo real, o item desaparece na mesma hora.

3.6 Favoritos e Perfil

- **Favoritar:** o coração da Tela 3 inverte o campo `favorito` do documento. A mudança aparece em todas as telas (o item na lista ganha um coração vermelho).
- **Aba Favoritos:** abre a Tela 2 já filtrada, mostrando só os favoritos de todas as cidades. O chip "Favoritos" faz o mesmo dentro da tela de resultados.
- **Aba Perfil:** abre um Bottom Sheet com o resumo do banco (total de locais, favoritos e cidades) e o status da conexão com o Cloud Firestore.

3.7 Fluxo de navegação

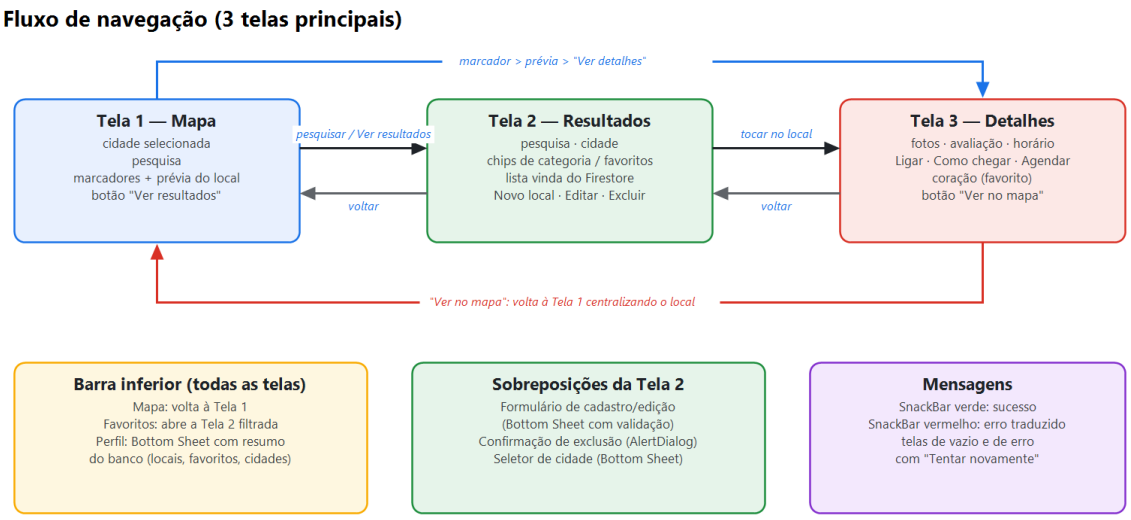


Figura 2 — Navegação entre as três telas e as janelas sobrepostas

A navegação usa `Navigator.push` para ir do mapa aos resultados e dos resultados aos detalhes, e `Navigator.pop` (botão voltar) para retornar. "Ver no mapa" usa `Navigator.popUntil` para voltar à primeira tela e avisa o mapa, por meio de um `ValueNotifier` compartilhado, qual local deve ser centralizado.

3.8 Mensagens e tratamento de erros

Toda operação no banco termina com uma mensagem (`utils/mensagens.dart`): **SnackBar verde** para sucesso ("Local cadastrado com sucesso!", "Adicionado aos favoritos!") e **SnackBar vermelha** para erro. Os erros mais comuns do Firestore são traduzidos para o usuário:

Código do Firestore	Mensagem exibida
`permission-denied`	Operação bloqueada pelas regras de segurança do Firestore.
`unavailable`	Sem conexão com o servidor. Verifique a internet.
`not-found`	O registro não existe mais no banco.
outros	Mensagem original do Firestore, com o código do erro.

3.9 Requisitos funcionais

Código	Requisito	Como foi atendido	Onde
RF01	Selecionar cidade	Seletor no mapa e filtro na lista (Limeira, São Paulo, Campinas, Jundiaí + "Todas")	`mapa_screen`, `seletor_cidade`
RF02	Pesquisar	Por nome ou categoria, sem acentos/maiúsculas	`utils/filtros.dart`
RF03	Consultar Firestore	Leitura em tempo real com `snapshots()`	`local_service.dart`

Código	Requisito	Como foi atendido	Onde
RF04	Filtrar	Categoria, cidade e favoritos	`resultados_screen.dart`
RF05	Cadastrar local	Formulário → `locais.add()`	`local_form.dart`
RF06	Editar local	Formulário preenchido → `doc(id).update()`	`resultados_screen.dart`
RF07	Excluir local	Confirmação → `doc(id).delete()`	`confirm_delete.dart`
RF08	Detalhes	Todas as informações do documento	`detalhes_screen.dart`
RF09	Favoritar	Atualiza o campo `favorito`	`detalhes_screen.dart`
RF10	Navegação	Mapa ⇄ Resultados ⇄ Detalhes, "Ver no mapa", barra inferior	`barra_navegacao.dart`

4. Conexões

Esta seção explica como o aplicativo se conecta a cada serviço externo: Firebase, Cloud Firestore, OpenStreetMap e aplicativos do Android.

4.1 Conexão com o Firebase

A configuração seguiu o fluxo oficial do Firebase para Flutter: **Firebase CLI + FlutterFire CLI + firebase_core + cloud_firestore**.

1. `firebase login` — autenticação da conta Google no Firebase CLI.
2. Criação do projeto `mapa-locais-gabriel` e do banco Cloud Firestore.
3. `flutterfire configure --project=mapa-locais-gabriel --platforms=android` — registrou o app Android no projeto e gerou os arquivos de configuração.
4. `flutter pub add firebase_core cloud_firestore` — pacotes do Firebase.

O FlutterFire CLI gerou e alterou estes arquivos:

Arquivo	Função
`lib/firebase_options.dart`	Identificadores do projeto (apiKey, appId, projectId…) usados por `Firebase.initializeApp`. Gerado automaticamente — não deve ser editado à mão.
`android/app/google-services.json`	Mesma configuração no formato do Android.
`android/settings.gradle.kts` e `app/build.gradle.kts`	Plugin `com.google.gms.google-services` (4.4.4), que lê o `google-services.json` no build.
`firebase.json` / `.firebaserc`	Projeto padrão, regras do Firestore e configuração do emulador local.

A conexão acontece no início do `main.dart`, antes de a interface ser desenhada:

```
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );

  if (usarEmulador) {   // --dart-define=USAR_EMULADOR=true
    FirebaseFirestore.instance.useFirestoreEmulator('10.0.2.2', 8080);
  }

  runApp(const MinhaAplicacao());
}
```

A `apiKey` presente em `firebase_options.dart` **não é uma senha**: ela apenas identifica o projeto. Quem protege os dados são as regras de segurança do Firestore (seção 4.4).

4.2 Leitura dos dados em tempo real

A leitura usa `snapshots()`, que mantém uma conexão aberta com o Firestore: sempre que um documento é criado, alterado ou excluído — pelo app ou pelo Console do Firebase —, o servidor envia a nova versão e a interface é redesenhada automaticamente por um `StreamBuilder`.

```
Stream<List<Local>> listarLocais() {
  return locais.snapshots().map((snapshot) {
    return snapshot.docs
        .map((doc) => Local.fromMap(doc.id, doc.data()))
        .toList();
  });
}
```

- **Mapa e Resultados** escutam `listarLocais()` (coleção inteira).
- **Detalhes** escuta `observarLocal(id)` (um único documento), o que atualiza o coração na hora e detecta se o local foi excluído.
- O SDK mantém um **cache local**: os dados já carregados continuam visíveis se a conexão cair, e as gravações aparecem na tela imediatamente, antes mesmo da confirmação do servidor.
- Pesquisa e filtros são aplicados **em memória** sobre a lista recebida (seção 5.3), como sugerido no tutorial.

4.3 Gravação dos dados (CRUD)

Todas as operações passam pela classe `LocalService` (`services/local_service.dart`). As telas nunca acessam o Firestore diretamente.

Operação	Comando no Firestore	Método do LocalService	Acionado por
Create	`locais.add(map)`	`adicionar(local)`	Botão "Novo local"
Read	`locais.snapshots()`	`listarLocais()` / `observarLocal(id)`	Mapa, Resultados, Detalhes
Update	`locais.doc(id).update(map)`	`atualizar(local)`	Menu ⋮ → Editar
Update parcial	`doc(id).update({'favorito': v})`	`alterarFavorito(id, v)`	Coração (Detalhes)
Delete	`locais.doc(id).delete()`	`excluir(id)`	Menu ⋮ → Excluir + confirmação
Create em lote	`batch.set(...)` + `commit()`	`popularExemplos()`	Botão "Cadastrar locais de exemplo"

```
Future<String> adicionar(Local local) async {
  final ref = await locais.add(local.toMap());   // ID gerado pelo Firestore
  return ref.id;
}

Future<void> atualizar(Local local) async {
  await locais.doc(local.id).update(local.toMap());
}

Future<void> excluir(String id) async {
  await locais.doc(id).delete();
```

```
}

Future<void> alterarFavorito(String id, bool favorito) async {
  await locais.doc(id).update({'favorito': favorito});
}
```

Nas telas, cada chamada fica dentro de `try/catch`: em caso de sucesso aparece a mensagem verde; em caso de falha, a mensagem vermelha traduzida (seção 3.8).

4.4 Segurança: regras do Firestore

O banco **não fica aberto**. As regras (`firestore.rules`) são executadas nos servidores do Google e validam cada requisição, mesmo que alguém tente acessar o banco sem usar o aplicativo:

Regra	Efeito
Qualquer coleção diferente de `locais`	Leitura e escrita **negadas**.
Leitura de `locais`	Pública — o app é um guia de estabelecimentos.
Cadastro e edição	Aceitos apenas se o documento seguir o modelo: somente campos conhecidos, campos obrigatórios presentes e tipos corretos.
Valores	Nome com 1 a 80 caracteres; categoria e cidade dentro das listas permitidas; avaliação de 0 a 5; distância de 0 a 500; limites de tamanho em horário, descrição, endereço e telefone; coordenadas válidas.

```
match /locais/{localId} {
  allow read: if true;
  allow create, update: if localValido(request.resource.data);
  allow delete: if true;
}
```

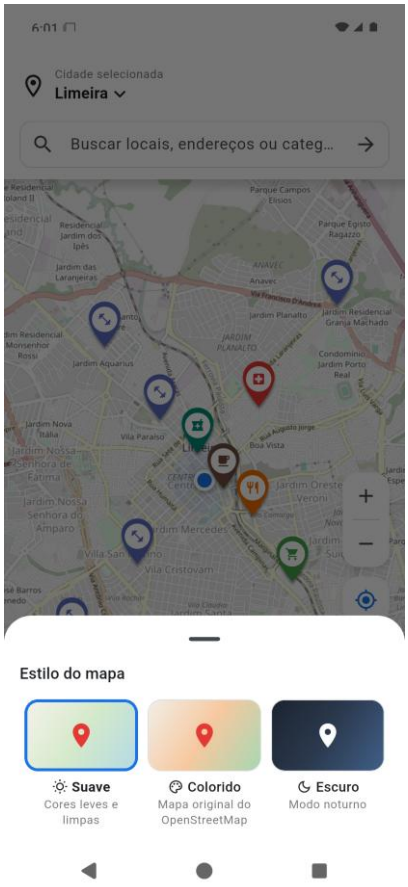
Teste das regras: no emulador local, um documento válido foi aceito (HTTP 200) e foram recusados (HTTP 403 — PERMISSION_DENIED) documentos com avaliação 9, com campo extra (`admin`), com cidade fora da lista, sem o campo `nome`, além de leitura e escrita em outra coleção. No projeto real, uma gravação com campo fora do modelo também retornou 403.

Limitação conhecida: sem login, qualquer pessoa ainda pode excluir um local. O próximo passo (desafio nível 3) é adicionar Firebase Authentication e exigir `request.auth` para cadastro, edição e exclusão.

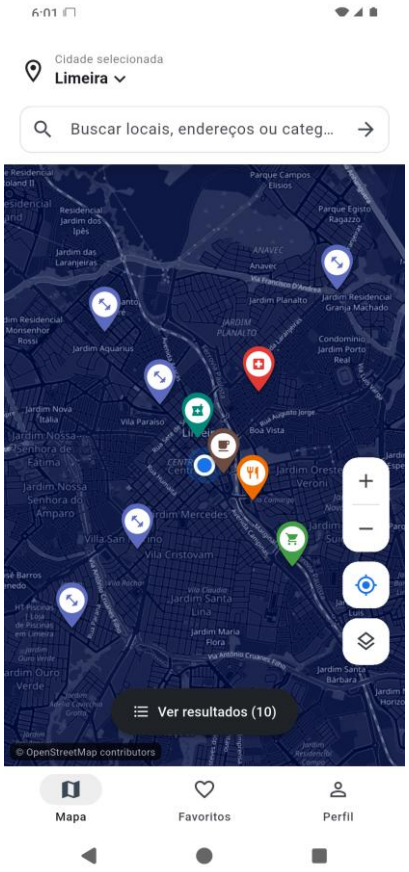
4.5 Mapa: OpenStreetMap

- O mapa usa o pacote `flutter_map`, que baixa as imagens ("tiles") de `tile.openstreetmap.org` por **HTTPS** e as monta conforme o zoom e a posição.
- As requisições se identificam com o nome do pacote do app (`userAgentPackageName`), como exige a política de uso do OpenStreetMap, e o mapa exibe a atribuição "OpenStreetMap contributors".

- **Estilos:** o Suave e o Escuro são filtros de cor (`ColorFilter.matrix`) aplicados às imagens no próprio aparelho, sem outro servidor. O Suave reduz a saturação e clareia (parques verdes e água azul continuam visíveis); o Escuro inverte as cores e puxa para azul-marinho; o Colorido mostra o mapa original.
- **Marcadores coloridos:** a cor e o ícone vêm do modelo `Categoria` (academia, restaurante, hospital, farmácia, mercado e café), o que ajuda a reconhecer o tipo de local de longe.
- Cada marcador é posicionado pela latitude e longitude gravadas no documento (`MarkerLayer`).
- Não é necessária chave de API. O Google Maps foi descartado porque exige uma conta de faturamento.



Seletor de estilo do mapa



Estilo Escuro (modo noturno)

4.6 Integrações com o Android

Recurso	Como funciona
Ligar	`url_launcher` abre o discador com `tel:` e o número do local.
Como chegar	Abre `https://www.google.com/maps/dir/?api=1&destination=lat,lng` no Google Maps ou no navegador.
Permissões (`AndroidManifest.xml`)	`INTERNET` (Firestore e mapa) e declarações `<queries>` para as intenções `DIAL` (tel) e `VIEW` (https), exigidas a partir do Android 11.

Recurso	Como funciona
Agendar e Compartilhar	Seletores de data e hora nativos do Material; o compartilhamento copia o texto para a área de transferência.

4.7 Ambiente de desenvolvimento

- **Emulador local do Firestore:** com `firebase emulators:start --only firestore` e `flutter run --dart-define=USAR_EMULADOR=true`, o app usa um banco que roda no próprio computador (endereço `10.0.2.2:8080`, que é o computador visto de dentro do emulador Android). Isso permitiu desenvolver e testar as regras antes de ligar o projeto real.
- **Rede da escola:** o emulador Android ficava sem internet porque o primeiro servidor DNS da rede não responde. O script `iniciar_emulador.bat` testa os DNS do computador e inicia o emulador com os que funcionam (`-dns-server`).
- **Acentos no caminho:** o projeto fica na pasta "Avaliação Formativa". O plugin Android recusa caminhos com acentos no Windows, por isso `android/gradle.properties` contém `android.overridePathCheck=true`. A análise de código é feita com `dart analyze`.

5. Arquitetura e organização do código

5.1 Camadas e responsabilidades

O código segue a estrutura recomendada no tutorial, com separação entre **Model**, **serviço** e **telas**. Isso isola o banco da interface e permitiu testar as telas com um Firestore falso.

Pasta / arquivo	Responsabilidade
`main.dart`	Inicializa o Firebase e abre o app.
`app.dart`	`MaterialApp`, tema visual (wireframe), idioma pt-BR e criação do `LocalService`.
`app_scope.dart`	`InheritedWidget` que entrega o `LocalService` e o "foco do mapa" a todas as telas (injeção de dependência).
`models/`	`Local` (fromMap/toMap/copyWith), `Cidade` (4 cidades e coordenadas; Limeira é a inicial) e `Categoria` (6 categorias, ícones e cores).
`services/local_service.dart`	Único ponto de acesso ao Firestore: CRUD, favoritos e carga de exemplos.
`screens/`	As três telas: `mapa_screen`, `resultados_screen`, `detalhes_screen`.
`widgets/`	Componentes reutilizáveis: card da lista, formulário, mapa, imagem ilustrativa, barra inferior.
`dialogs/`	Confirmação de exclusão, seletor de cidade e aba Perfil.
`utils/`	Filtros e pesquisa, formatação (km, nota, horário aberto/fechado), coordenadas e mensagens.
`data/locais_exemplo.dart`	Os 20 locais usados para popular o banco.

Fluxo de uma operação: a tela chama um método do `LocalService` → o serviço converte o `Local` em `Map` (`toMap`) e usa o `cloud_firestore` → o Firestore aplica as regras e grava → o `snapshots()` devolve a nova lista → o `StreamBuilder` redesenha as telas. O projeto tem cerca de **2.900 linhas** de código no app e **580** nos testes.

5.2 Modelo de dados — coleção locais

Campo	Tipo	Obrigatório	Exemplo
nome	string	sim	"Academia Fitness"
categoria	string	sim	"Academia"
cidade	string	sim	"Limeira"
avaliacao	number (0–5)	sim	4.8

Campo	Tipo	Obrigatório	Exemplo
distancia	number (km)	sim	1.2
horario	string	sim	"06:00 - 22:00"
descricao	string	sim	"Academia completa com musculação..."
favorito	boolean	sim	false
endereco	string	não	"Rua Senador Vergueiro, 1100 - Centro"
telefone	string	não	"(19) 3400-1003"
latitude / longitude	number	não	-22.5598 / -47.4066

Os oito primeiros campos são os definidos no tutorial. Os campos complementares alimentam a tela de detalhes e o mapa. O **ID do documento** é gerado pelo Firestore e guardado no campo `id` do Model, para identificar o documento na edição, na exclusão e no favorito. A conversão é feita por `Local.fromMap` e `toMap`; números inteiros vindos do banco são convertidos para `double`.

5.3 Pesquisa e filtros

A função `filtrarLocais` (`utils/filtros.dart`) recebe a lista vinda do Firestore e aplica quatro critérios ao mesmo tempo: **cidade** ("Todas" ou igual), **categoria** ("Todos" ou igual), **texto** (contido no nome ou na categoria) e **apenas favoritos**. Antes de comparar, o texto passa por `normalizar`, que converte para minúsculas e remove acentos. O resultado é ordenado pela distância.

6. Testes

6.1 Estratégia

Tipo	Arquivo	Qtde.	O que verifica
Unidade	`test/local_test.dart`	3	Conversões do Model (`fromMap`, `toMap`, valores padrão).
Unidade	`test/filtros_test.dart`	10	Pesquisa, filtros, ordenação, dados de exemplo, horário aberto/fechado.
Unidade	`test/local_service_test.dart`	6	CRUD do `LocalService` contra um Firestore em memória.
Widget	`test/widget_test.dart`	13	Fluxos das telas (T01–T11) e seletor de estilo do mapa, com Firestore em memória.
Integração	`integration_test/app_test.dart`	1	Fluxo completo no emulador Android, no **Firestore real**: pesquisa, cidade, filtro, cadastro, leitura, favorito, edição e exclusão.

Resultado (28/10/2026): os 32 testes de unidade e de widget e o teste de integração passaram no emulador **Pixel 8 (Android 16)** conectado ao projeto `mapa-locais-gabriel`. O teste de integração cria, edita, favorita e exclui um local próprio, sem alterar os outros documentos.

6.2 Testes obrigatórios

ID	Teste	Procedimento	Resultado
T01	Conexão	Abrir o app e verificar se os dados do Firestore aparecem	Passou
T02	Pesquisa	Pesquisar por "Academia"	Passou
T03	Cidade	Selecionar Campinas	Passou
T04	Filtro	Selecionar uma categoria	Passou
T05	Create	Cadastrar um novo local e verificar no Firestore	Passou
T06	Read	Fechar/reabrir a tela e verificar o registro	Passou
T07	Update	Editar nome ou horário e confirmar a alteração	Passou
T08	Delete	Excluir um local e confirmar que desapareceu	Passou
T09	Favorito	Marcar e desmarcar um local	Passou
T10	Navegação	Abrir resultados e detalhes e retornar	Passou
T11	Erro	Simular ausência de dados/erro e verificar mensagem	Passou

T01 a T10 são cobertos pelo teste de integração no Firestore real e pelos testes de widget. O T11 é verificado pelos testes de widget com um banco vazio (mensagem "Nenhum local cadastrado." e botão de exemplos) e com uma falha de conexão simulada (mensagem "Erro ao carregar os locais." e "Tentar novamente").

6.3 Evidência no Firestore

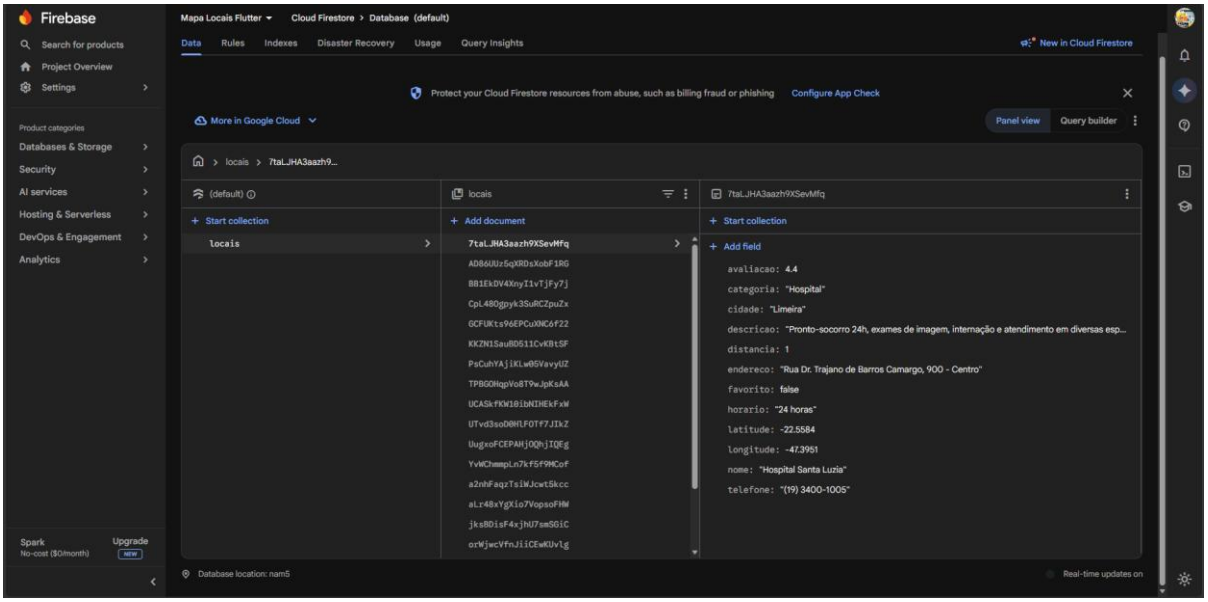


Figura 3 — Console do Firebase: coleção locais com os documentos e os campos de um local

7. Instalação e execução

7.1 Pré-requisitos

- Flutter 3.47 (Dart 3.13) e Android SDK com um emulador (Pixel 8, Android 16).
- Node.js com Firebase CLI (`npm install -g firebase-tools`) e FlutterFire CLI (`dart pub global activate flutterfire_cli`) — apenas para reconfigurar o Firebase.

7.2 Executar o app

1. Abrir o emulador com `iniciar_emulador.bat` (corrige o DNS da rede da escola).
2. Na pasta `mapa_locais`: `flutter pub get`.
3. `flutter run`.
4. Se a coleção estiver vazia, usar "Cadastrar locais de exemplo" na tela de resultados.

7.3 Comandos úteis

```
flutter test                                    # 32 testes de unidade e widget
flutter test integration_test -d emulator-5554  # teste de ponta a ponta no Firestore real
dart analyze                                    # análise estática do código
firebase deploy --only firestore:rules          # publicar as regras de segurança

# desenvolvimento sem o banco real:
firebase emulators:start --only firestore --project demo-mapa-locais
flutter run --dart-define=USAR_EMULADOR=true
```

8. Requisitos não funcionais, limitações e conclusão

8.1 Requisitos não funcionais atendidos

- Desenvolvido em Flutter/Dart e executando em Android.
- Banco Cloud Firestore configurado com FlutterFire.
- Código organizado em pastas (models, services, screens, widgets, dialogs, utils, data).
- Exclusões sempre pedem confirmação.
- Mensagens de sucesso e de erro em todas as operações de banco.
- Banco protegido por regras de segurança, sem acesso aberto.
- Sem erros na análise estática (`dart analyze`) e com todos os testes passando.

8.2 Limitações e melhorias futuras

Limitação atual	Melhoria proposta
Sem login: qualquer pessoa pode cadastrar/editar (dados validados) e excluir.	Firebase Authentication e regras com `request.auth`, separando usuários e administradores (nível 3).
Favoritos são um campo do local, iguais para todos os usuários.	Lista de favoritos por usuário em uma subcoleção (nível 5).
Imagens são ilustrações por categoria.	Fotos reais com Firebase Storage (nível 1).
Distância é um valor gravado; a posição do usuário é simulada no centro da cidade.	GPS real (pacote geolocator) e cálculo da distância pelas coordenadas.
Filtros aplicados em memória.	Consultas com `where()`/`orderBy()` e índices no Firestore (nível 4).

Desafio extra implementado — nível 2 (mapa real): o mapa usa o OpenStreetMap com três estilos (Suave, Colorido e Escuro) e marcadores coloridos por categoria, posicionados pelas coordenadas de cada documento.

8.3 Conclusão

O aplicativo atende ao wireframe e aos requisitos da avaliação: três telas, dados no Cloud Firestore, pesquisa, filtros, CRUD completo, favoritos e navegação. A separação entre Model, serviço e telas deixou o código simples de manter e de testar, e o uso de `snapshots()` tornou a interface reativa: qualquer alteração no banco aparece imediatamente em todas as telas.

O desenvolvimento por etapas — primeiro com o emulador local do Firestore e depois com o projeto real — permitiu validar as regras de segurança antes de publicá-las. O projeto também mostrou que um banco em nuvem precisa ser protegido: mesmo sem login, as regras impedem dados fora do modelo e o acesso a outras coleções. O próximo passo natural é a autenticação de usuários.

Referências

- Firebase para Flutter: https://firebase.google.com/docs/flutter/setup
- Cloud Firestore — adicionar e gerenciar dados: https://firebase.google.com/docs/firestore/manage-data/add-data
- Regras de segurança do Cloud Firestore: https://firebase.google.com/docs/firestore/security/get-started
- flutter_map: https://docs.fleaflet.dev
- Política de uso dos tiles do OpenStreetMap: https://operations.osmfoundation.org/policies/tiles/